The Cereal Selection Puzzle: PCA Unmasks Healthy Cereals

Liran Ma, Zhipeng Cai, and Rongrong Qian

Introduction: The Cereal Selection Puzzle

Figure 1: The challenge: selecting truly healthy cereals from confusing nutrition labels.

Picture this: Alex is scanning the cereal aisle at Kroger, determined to pick the healthiest option. He spots:

- **Cereal A:** Sugar 5 g, Fat 3 g, Fiber 8 g.....
- **Cereal B:** Sugar 3 g, Fat 8 g, Fiber 2 g.....
- **Cereal C:** Sugar 12 g, Fat 2 g, Fiber 10 g.....

Following conventional wisdom, Alex selects **Cereal B** with the lowest sugar content (3 g). After consuming it for a week, Alex is surprised to gain body fat!

(Increased body fat is associated with higher risks of chronic diseases. This means that even though Alex ate less sugar, his overall health may have gotten worse, as shown by a lower health score.)

The Puzzle: Why did the "low-sugar" choice lead to unexpected results?

Sugar alone isn't enough to judge a cereal's healthiness. Fat, fiber, protein, and other nutrients also matter. But with so many variables, how do we decide which ones are most important? That's where principal component analysis (PCA) can help.

We'll solve this mystery using PCA with the 80 Cereals Dataset from Kaggle.

1 Theoretical Foundations

1.1 The Challenge: Multidimensional Nutrition Space

Cereal nutrition involves multiple dimensions:

$$\text{Features: } \{\text{calories, protein, fat, sodium, fiber, carbo, sugars, potass}\}$$
$$\text{Target: rating (health score)}$$

Working with high-dimensional data can be challenging. Not only is it difficult to visualize and interpret, but it also requires more computational resources. In real-world datasets like cereal nutrition, we often face many variables (calories, protein, fat, sodium, fiber, etc.), but analyzing them all together can be complex and overwhelming.

To make analysis easier and more efficient, we need to reduce the number of dimensions in our data, while still retaining as much information as possible. Principal Component Analysis (PCA) achieves this by combining the original variables into a smaller set of new variables, called principal components (PC1, PC2, ...). Each principal component is a weighted combination of all the original variables, chosen so that together, they explain most of the variation in the data.

In other words, PCA transforms our high-dimensional data into a lower-dimensional space, capturing the main patterns and relationships in just a few components.

1.2 PCA Illustration with Two Variables

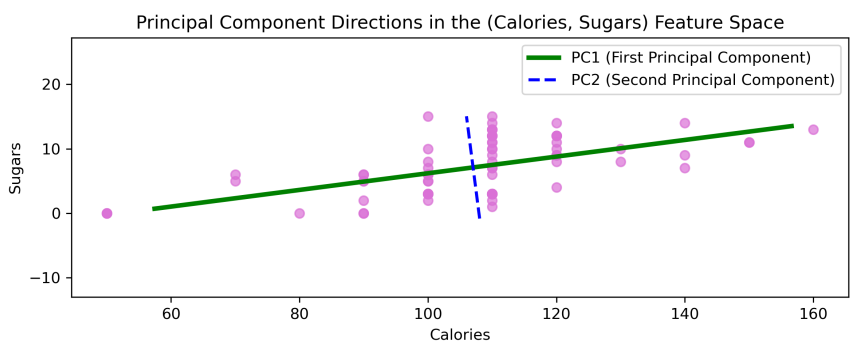

Figure 2: Original (Calories, Sugars) Space. PC1 (green) and PC2 (blue) show the directions capturing the most and second-most variance in the data.

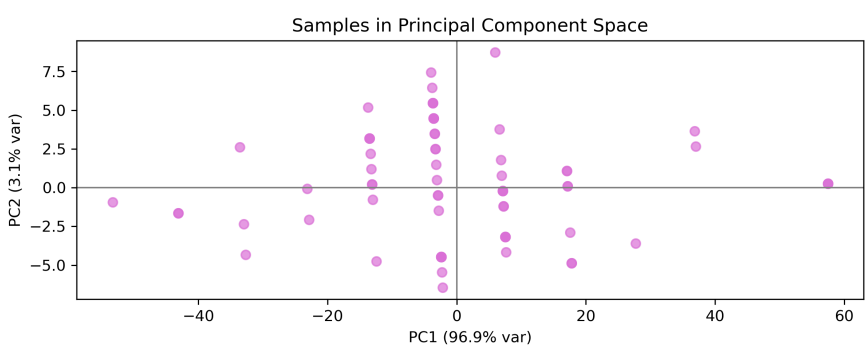

Figure 3: Principal Component Space. All samples are re-expressed as coordinates on the PC1 and PC2 axes (the new basis found by PCA).

Understanding Principal Components in Two Dimensions:

In this example, we plot cereals in the (Calories, Sugars) space. The first principal component (PC1), shown as a green line, is the direction along which the data vary the most. You can think of PC1 as the line that best summarizes the spread of the data. It captures the greatest amount of information (variance). In fact, finding PC1 is mathematically similar to fitting a line that "predicts" Sugars from Calories using linear regression, except PCA is unsupervised and considers both axes equally.

Once PC1 is found, the second principal component (PC2) is determined. PC2 is always perpendicular to PC1 and captures the largest remaining variance that was not already explained by PC1. In other words, PC2 is the second-most informative direction in the data.

Together, PC1 and PC2 define a new coordinate system that can describe the original data with maximum efficiency. This process generalizes to higher dimensions, where the first few principal components capture the bulk of the data's structure.

Question: In higher-dimensional PCA, what is happening when we extract principal components?

1.3 How does PCA work?

To perform PCA, we follow these key steps:

1. **Standardize the data:** Center each feature to mean zero and scale to unit variance, so that all variables are comparable.
2. **Compute the covariance matrix:** Calculate how each pair of variables varies together, capturing the relationships between features.
3. **Compute eigenvalues:** Show the variance captured by each principal component.
4. **Compute eigenvectors (PCA loadings):** Examine the contribution of each original feature to each principal component.
5. **Show loadings for PC1 & PC2:** Focus on the main components that capture the most variance.
6. **Compute explained variance ratios:** Display how much variance each principal component explains.
7. **Project the data:** Transform the original data onto the new lower-dimensional space defined by the selected principal components.

Each of these steps helps reduce the complexity of the data while keeping the main patterns and relationships.

1.4 Key Mathematical Concepts

- **Covariance Matrix:** Captures feature relationships.

$$\mathrm{Cov}(X, Y) = \frac{\sum_{i=1}^{n}(X_i - \bar{X})(Y_i - \bar{Y})}{n-1}.$$

- **Eigenvectors and Eigenvalues:** Directions of maximum variance and their corresponding magnitude.

$$A\,v = \lambda\,v.$$

  Here, A is the covariance matrix, v is an eigenvector (principal component direction), and λ is the corresponding eigenvalue (the amount of variance explained by v). Each eigenvector points to a direction in feature space, and its eigenvalue tells us how much variance is captured along that direction.

- **Explained Variance:** Fraction of total variance captured by each principal component.

$$\text{Explained Variance} = \frac{\lambda_i}{\sum_{j=1}^{k} \lambda_j} \times 100\%.$$

  The larger the eigenvalue λ_i, the more important its principal component.

- **Principal Component Scores:** Each principal component score is a linear combination of the original features:

$$z_{im} = \sum_{j=1}^{p} \phi_{jm} X_{ij}$$

  where ϕ_{jm} are the loadings (elements of the eigenvector for PC m), and X_{ij} is the value of feature j for observation i.

  Example: Suppose we have a cereal sample with standardized (Calories, Sugars) values $(x_1, x_2) = (1.2,\ -0.5)$. If the first principal component (PC1) is defined by the loading vector $\phi_1 = (0.7,\ 0.7)$, then the PC1 score for this sample is:

$$z_{i1} = 0.7 \times 1.2 + 0.7 \times (-0.5) = 0.84 - 0.35 = 0.49$$

  So, $z_{i1} = 0.49$ is the coordinate of this sample along the PC1 axis.

  Exercise: Suppose for a cereal sample, the standardized values are $(x_1, x_2) = (0.8,\ 1.5)$. If the first principal component (PC1) is defined by the loading vector $\phi_1 = (0.6,\ 0.8)$, what is the PC1 score z_{i1} for this sample?

  The PC1 score z_{i1} for this sample is: _____

- **Linear Regression after Dimension Reduction:** After projecting data to the first M principal components (z_{im}), we fit a linear regression model:

$$y_i = \theta_0 + \sum_{m=1}^{M} \theta_m z_{im} + \epsilon_i, \qquad i = 1, \ldots, n.$$

  - y_i: The response value (e.g., health rating) for the ith sample.
  - θ_0: Intercept term.
  - θ_m: Regression coefficient for the mth principal component.
  - z_{im}: The PCm score for the ith sample.
  - ϵ_i: The residual (error) for the ith sample.

2 Data Preparation and Exploratory Data Analysis (EDA)

2.1 Data Cleaning

The 80 Cereals Dataset contains nutritional information for 80 popular cereals:

- 8 nutritional features,
- 1 response variable (health rating),
- Manufacturer information.

We first read the dataset and remove all rows with missing values or placeholder values (-1) in any of the main columns.

```
import numpy as np
import pandas as pd

# 1. Read data
df = pd.read_csv("cereal.csv")

# 2. Select features and target, remove missing values
feature_cols = ['calories', 'protein', 'fat', 'sodium', 'fiber', 'carbo', 'sugars'
    , 'potass']
target_col   = 'rating'

df_clean = df.loc[:, ['name'] + feature_cols + [target_col]] \
             .replace(-1, np.nan) \
             .dropna(axis=0, how='any') \
             .reset_index(drop=True)

X_df  = df_clean[feature_cols].copy()
y     = df_clean[target_col].values
names = df_clean['name'].values

print(f"Number of cereals after cleaning: {len(df_clean)}")
```

2.2 Correlation Analysis

We compute and display the correlation matrix to reveal relationships among the nutritional features.

```
print("=== Correlation Matrix of Features ===")
print(X_df.corr())
```

```
=== Correlation Matrix ===
          calories   protein       fat    sodium     fiber     carbo  \
calories  1.000000  0.033992  0.507373  0.296247 -0.295212  0.270606
protein   0.033992  1.000000  0.202353  0.011559  0.514006 -0.036743
fat       0.507373  0.202353  1.000000  0.000822  0.014036 -0.284934
sodium    0.296247  0.011559  0.000822  1.000000 -0.070735  0.328409
fiber    -0.295212  0.514006  0.014036 -0.070735  1.000000 -0.379084
carbo     0.270606 -0.036743 -0.284934  0.328409 -0.379084  1.000000
sugars    0.569121 -0.286584  0.287152  0.037059 -0.150949 -0.452069
potass   -0.071361  0.578743  0.199637 -0.039438  0.911504 -0.365003

            sugars    potass
calories  0.569121 -0.071361
protein  -0.286584  0.578743
fat       0.287152  0.199637
sodium    0.037059 -0.039438
fiber    -0.150949  0.911504
carbo    -0.452069 -0.365003
sugars    1.000000  0.001414
potass    0.001414  1.000000
```

Figure 4: Correlation matrix revealing nutritional relationships.

Key Findings from the Correlation Matrix:

- **Sugars and Calorie have $r = 0.569$.**
- **Fat and Calorie** _____
- **Fiber and Protein** _____
- **Potassium and Protein** _____
- **Potassium and Fiber** _____

Alex's Selection and Preliminary Findings:
If Alex chose "Cheerios" for its low sugar (1 g), but it has high fat (2 g) and its health rating is only 33.98. Judging health by sugar alone can mislead, since other nutrients (like fat) strongly influence overall health.

3 PCA Implementation: Unveiling Hidden Structures

Before applying PCA, replace any `-1` entries with NaN and drop rows containing missing values. After cleaning, standardize the eight nutritional features to mean zero, unit variance, then compute a full PCA (8 components).

3.1 Data Standardization and Full PCA

```python
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

# X_df is the clean DataFrame of shape (74, 8) after dropping all -1 rows.
scaler = StandardScaler()
X_std  = scaler.fit_transform(X_df)    # (74, 8) standardized features

```

```python
pca_full = PCA(n_components=8)
pca_full.fit(X_std)

print("Eigenvalues (PCA full):")
print(pca_full.explained_variance_, "\n")
print("\nExplained Variance Ratios:")
for i, ratio in enumerate(pca_full.explained_variance_ratio_, start=1):
    print(f"PC{i}: {ratio:.4f} ({ratio*100:.1f}%)")
```

Listing 1: Fit full PCA on all 8 features

Results of the Full PCA:

- **Eigenvalues:**

$$\lambda = [2.6640,\ 2.0484,\ 1.5255,\ 0.8757,\ 0.5426,\ 0.3742,\ 0.0597,\ 0.0194].$$

- **Explained Variance Ratios:**

$$\begin{aligned}
&\text{PC1} : 0.3285\,(32.9\%),\\
&\text{PC2} : 0.2526\,(25.3\%),\\
&\text{PC3} : 0.1881\,(18.8\%),\\
&\text{PC4} : 0.1080\,(10.8\%),\\
&\text{PC5} : 0.0669\,(6.7\%),\\
&\text{PC6} : 0.0461\,(4.6\%),\\
&\text{PC7} : 0.0074\,(0.7\%),\\
&\text{PC8} : 0.0024\,(0.2\%).
\end{aligned}$$

The first two components together account for 58.2% of total variance.

3.2 PCA Loadings (Eigenvectors)

```python
# Construct a DataFrame of loadings (features x PCs):
loadings = pd.DataFrame(
    pca_full.components_.T,
    index=feature_cols,
    columns=[f"PC{i+1}" for i in range(8)]
)
print(loadings.loc[:, ["PC1", "PC2"]])
```

Listing 2: Print loadings for the first two PCs

Loadings (features vs. PC1 and PC2):

Feature	PC1	PC2
calories	−0.1977	−0.5539
protein	0.4127	−0.0114
fat	0.1048	−0.5284
sodium	−0.1389	−0.1026
fiber	0.5730	0.0483
carbo	−0.3239	0.2442
sugars	−0.0891	−0.5734
potass	0.5649	−0.1121

- **PC1**:

$$+\,\text{fiber}\,(0.5730)\;+\;\text{potass}\,(0.5649)\;+\;\text{protein}\,(0.4127)$$
$$-\,\text{calories}\,(0.1977)\;-\;\text{carbo}\,(0.3239)\;-\;\text{sugars}\,(0.0891)\,.$$

  Cereals with large PC1 scores are fiber-/protein-/potassium-rich and lower in sugar/carbs—i.e. nutrient-dense.

- **PC2**:

$$-\,\text{sugars}\,(0.5734)\;-\;\text{calories}\,(0.5539)\;-\;\text{fat}\,(0.5284)$$
$$+\,\text{carbo}\,(0.2442)\;+\;\text{fiber}\,(0.0483)\,.$$

  Negative PC2 indicates high sugar/calories/fat—i.e. an unhealthy direction.

3.3 Dimensionality Reduction to Two Components

```
pca2   = PCA(n_components=2)
X_pca2 = pca2.fit_transform(X_std)  # (74, 2)
```

Listing 3: Compute 2D PCA scores

Now $X_{\text{pca2}} \in \mathbb{R}^{74\times 2}$, where

$$X_{\text{pca2}}[i] = \big(\text{PC1}_i,\ \text{PC2}_i\big), \quad i = 1, \ldots, 74.$$

3.4 PCA Biplot (PC1 vs. PC2)

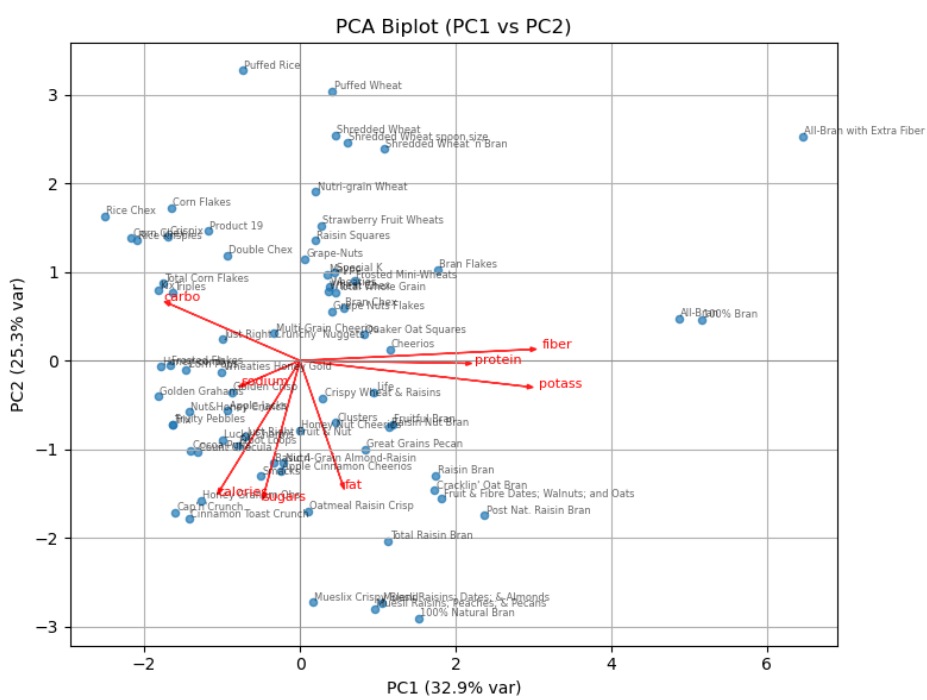

Figure 5: PCA Biplot: each point is a cereal in (PC1, PC2) space; red arrows show original feature loadings.

How to Interpret the PCA Biplot:

This PCA biplot displays two main elements:

- **Points (blue):** Each point represents a cereal brand, plotted using its coordinates on the first two principal components (PC1 and PC2). These axes capture the greatest variance in the data, so points that are close together have similar nutritional profiles, while points far apart are more different.
- **Red arrows (loadings):** The red arrows indicate how the original nutritional variables (like fiber, protein, sugars, etc.) contribute to the principal components:
  - The **direction** of an arrow shows which way in PC space that nutrient increases.
  - The **length** of an arrow reflects how strongly that nutrient influences the variation captured by PC1 and PC2.
  - For example, if "fiber" and "protein" arrows point in a similar direction, it means cereals high in fiber also tend to be high in protein.

4 Model Comparison: The Power of PCA

We compare four linear-regression models, each fit to the entire dataset of 74 cereals. We report R^2, MAE, and Alex's prediction error (Alex = *All-Bran with Extra Fiber*, true rating = 93.70).

4.1 Python Code

```python
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error

# 1) Full Model (all 8 features)
model_full    = LinearRegression().fit(X_df.values, y)
y_pred_full   = model_full.predict(X_df.values)

# 2) Sugar Only Model
X_sugar       = X_df[['sugars']].values.reshape(-1,1)
model_sugar   = LinearRegression().fit(X_sugar, y)
y_pred_sugar  = model_sugar.predict(X_sugar)

# 3) PCA Model (PC1 + PC2)
X_pc          = X_pca2
model_pca     = LinearRegression().fit(X_pc, y)
y_pred_pca    = model_pca.predict(X_pc)

# 4) PC1-Only Model
X_pc1         = X_pca2[:,0].reshape(-1,1)
model_pc1     = LinearRegression().fit(X_pc1, y)
y_pred_pc1    = model_pc1.predict(X_pc1)
```

Listing 4: Fit and evaluate all four regression models

4.2 Alex's Predictions

$$
\begin{aligned}
\text{Alex's true rating} &= 93.70, \\
\widehat{y}^{\text{Full}}_{\text{Alex}} &= 93.93, \quad |93.70 - 93.93| = 0.22, \\
\widehat{y}^{\text{Sugar}}_{\text{Alex}} &= 59.67, \quad |93.70 - 59.67| = 34.03, \\
\widehat{y}^{\text{PCA}}_{\text{Alex}} &= 94.15, \quad |93.70 - 94.15| = 0.45, \\
\widehat{y}^{\text{PC1}}_{\text{Alex}} &= 75.64, \quad |93.70 - 75.64| = 18.07.
\end{aligned}
$$

4.3 R^2 and MAE for Each Model

Model	R^2	MAE
Full Model	0.9946	0.6112
Sugar Only	0.5715	7.0878
PCA Model	0.9165	3.0742
PC1-Only	0.3585	9.2581

4.4 Regression Equations

$$\text{PCA Model: } \widehat{y} = 5.148\,(\text{PC1}) + 7.325\,(\text{PC2}) + 42.372,$$

$$\text{PC1-Only: } \widehat{y} = 5.148\,(\text{PC1}) + 42.372.$$

Key Takeaways:

- The **PCA Model** (two components) achieves $R^2 = 0.9165$, MAE ≈ 3.0742, nearly matching the Full Model ($R^2 = 0.9946$, MAE ≈ 0.6112). Alex's error is 0.45 vs. 0.22 from the Full Model.
- The **PC1-Only** model has $R^2 = 0.3585$, MAE ≈ 9.2581, and Alex's error = 18.07, confirming that two PCs are necessary for strong predictive power.

4.5 Top-5 Predictions

Table 1: Top 5 cereals by predicted rating under PC1-Only vs. PCA models

Rank	PC1-Only		PCA (PC1+PC2)	
	Name	Pred_PC1	Name	Pred_PCA
1	All-Bran with Extra Fiber	75.64	All-Bran with Extra Fiber	94.15
2	100% Bran	68.92	100% Bran	72.33
3	All-Bran	67.46	All-Bran	70.96
4	Post Nat. Raisin Bran	54.58	Puffed Wheat	66.75
5	Fruit & Fibre Dates; Walnuts; and Oats	51.75	Shredded Wheat 'n' Bran	65.43

Interpretation: Both PC1-Only and PCA models rank "All-Bran with Extra Fiber," "100% Bran," and "All-Bran" at the top. The PCA model's predicted ratings align more closely with true ratings due to its lower MAE.

5 Conclusion

We have demonstrated:

1. **Data Cleaning & EDA:** Correlations confirm that sugar, fat, and calories are strongly intertwined; fiber and protein cluster together.
2. **PCA:** The first two components explain 58.2% of total variance. PC1 captures "nutrient-dense" (fiber, protein, potassium), while PC2 captures "unhealthy" (sugar, calories, fat).

3. **Model Comparison:** PCA(2) achieves $R^2 = 0.9165$, MAE ≈ 3.0742, nearly matching the Full Model ($R^2 = 0.9946$, MAE ≈ 0.6112), using only two features.

4. **Actionable Guidance:** Avoid "sugar-only" heuristics, emphasize fiber, protein, and potassium. Use PCA to reveal hidden correlations.

Alex's key mistake was focusing on a single dimension (sugar), whereas PCA's multidimensional view shows that sugar and fat travel together along an "unhealthy axis." Once that is revealed, selecting truly healthy cereals becomes obvious.

"PCA doesn't just reduce dimensions, it reveals the hidden structure in your data."